WNDRVR
SUPPORT NETWORK

LEARNING BRIDGE PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

Learning Bridge Programmer's Guide, SR0650

10 June 2020

1. OVERVIEW

1. Learning Bridge for VxWorks

Learning Bridge Programmer's Guide describes how VxWorks® is a transparent layer-two Ethernet learning bridge.

The Learning Bridge builds a database called a station cache. The station cache stores the relationships between MAC addresses and the ports where the MAC addresses came from. The bridge uses its station cache as the basis for traffic forwarding. For more information about the station cache, see Learning Bridge Station Cache Parameters on page 10 .

Transparent bridges are so named because their presence and operation are transparent to network hosts. Transparent bridges learn the network's topology by analyzing the source addresses of incoming frames from all attached networks.

Source Code Files

The source code files for the learning bridge are located within the linkproto layer in the following directory:

installDir **/vxworks-7/pkgs_v2/net/ipnet/linkproto/bridge/**

About the IP Addresses Used in This Manual on page 1IP addresses for networking examples follow the guidelines provided by RFC 1918.

Next topic: About the Learning Bridge Technology on page 1

1.1. About the IP Addresses Used in This Manual

IP addresses for networking examples follow the guidelines provided by RFC 1918.

According to RFC 1918, the Internet Assigned Numbers Authority (IANA) has reserved the following three blocks of the IPv4 address space for private networks:

- 10.0.0.0 - 10.255.255.255 (10/8 prefix)
- 172.16.0.0 - 172.31.255.255 (172.16/12 prefix)
- 192.168.0.0 - 192.168.255.255 (192.168/16 prefix)

These address spaces are useful in networking examples, which need to function but also need to avoid public Internet addresses.

In this guide, the 10.0.0.0/8 address space, the largest of the three private address spaces, is used as a stand-in for the public Internet. To indicate a private address space, this guide uses the 192.168/16 address space.

For IPv6 addresses, Wind River uses the 2001:DB8::/32 address space. This space is reserved for test and documentation addresses.

Parent topic: Learning Bridge for VxWorks on page 1

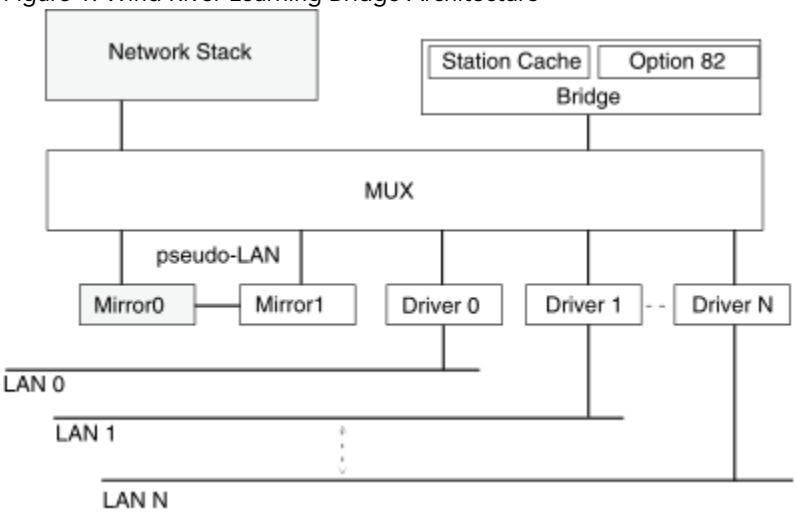
2. About the Learning Bridge Technology

Learning Bridge includes a bridge that attaches above the multiplexer (MUX) layer as a network service of type SNARF, and two Mirror enhanced network (END) drivers that attach below the MUX layer.

WNDRVR

The SNARF protocol or service sees every packet that is processed by a driver to which the protocol or service is bound. The two Mirror END drivers simulate a LAN that the bridge uses to handle traffic destined for a stack residing on the same machine as the bridge.

Figure 1. Wind River Learning Bridge Architecture



Note: In the figure above, **Mirror0** is not a bridge port and is attached to the network stack.

An arbitrary number of END or network protocol toolkit (NPT) drivers can be attached as bridge ports. All of the network interfaces that need to be a part of the transparent bridge must have promiscuous mode enabled so that the transparent bridge receives all the packets the network interfaces see.

The setup illustrated in the figure above shows these components along with the network stack and multiple network drivers connecting to multiple LANs.

Learning Bridge Mirror END Driver on page 2 The mirror enhanced network (END) driver behaves much like a standard END driver.
Learning Bridge Relay Agent Information Option (option 82) on page 3 The relay agent information option (option 82) feature enables dynamic host control protocol (DHCP) servers to implement security and IP address assignment policies based on information received by a bridge or relay agent.
Learning Bridge Driver Requirements on page 3 For a network driver to work with the learning bridge architecture, it must be written as either a mirror enhanced network (END) driver or a network protocol toolkit (NPT) driver (in other words, a driver that uses the MUX).

Previous topic: Learning Bridge for VxWorks on page 1
Next topic: Learning Bridge Source Address Processing on page 3

2.1. Learning Bridge Mirror END Driver

The mirror enhanced network (END) driver behaves much like a standard END driver.

The driver is installed as a pair of drivers in the system, typically with the device names **Mirror0** and **Mirror1**. Anything sent to **Mirror0** is received at **Mirror1** and anything sent to **Mirror1** is received at **Mirror0**.

Mirror0 is attached to the network stack and **Mirror1** is attached to the bridge. In order for **Mirror0** to function properly with the network stack, it must have a MAC address (even though it does not actually have underlying hardware). The MAC address provided for **Mirror0** is the MAC address of the first port of the **BRIDGE_PORTS** parameter of the bridge component. **Mirror1** is set to operate in promiscuous mode and therefore does not require a MAC address.

Parent topic: About the Learning Bridge Technology on page 1

2.2. Learning Bridge Relay Agent Information Option (option 82)

The relay agent information option (option 82) feature enables dynamic host control protocol (DHCP) servers to implement security and IP address assignment policies based on information received by a bridge or relay agent.

The DHCP server can also use this information to perform access control and set quality of service (QoS) for each client.

For more information on the relay agent information option (option 82), see Learning Bridge DHCP Relay Agent Information Option (Option 82) on page 6.

Parent topic: About the Learning Bridge Technology on page 1

2.3. Learning Bridge Driver Requirements

For a network driver to work with the learning bridge architecture, it must be written as either a mirror enhanced network (END) driver or a network protocol toolkit (NPT) driver (in other words, a driver that uses the MUX).

In addition, it must use Ethernet media access control (MAC) framing. This framing can be of either the Ethernet II or the IEEE 802.3 variety.

Your driver is allowed to send and receive Ethernet frames inside of additional encapsulation, if the driver can remove the additional encapsulation when it communicates with the MUX.

Parent topic: About the Learning Bridge Technology on page 1

3. Learning Bridge Source Address Processing

When a frame is received on a learning bridge interface, the bridge checks to see if the source media access control (MAC) address is in the station cache.

If the source MAC address is not in the station cache, the bridge creates a new station cache entry, linking the source MAC address with its port. If the source MAC address is in the station cache, but this MAC address is now connected to a different port, the bridge updates the station cache. If the source MAC address is in the station cache, the bridge resets the station cache entry's tick count.

> Note: The static entries in the station cache are not modified; they are permanent. A static entry represents a permanent mapping between the source MAC address and a bridge port.

Previous topic: About the Learning Bridge Technology on page 1
Next topic: Learning Bridge Destination Address Processing on page 3

4. Learning Bridge Destination Address Processing

When the Learning Bridge receives a packet at the destination address, it checks the destination media access control (MAC) address and the station cache to detemine what to do next.

If it is a broadcast address, the bridge forwards the packet to all ports except for the source port. If the destination MAC address is not a broadcast MAC address, the bridge looks up the frame's destination address in its station cache.

WNDRVR

If the station cache contains an association between the destination address and any of the bridge's ports, the frame is forwarded out the associated port. If the destination port is the same as the source port on which the frame was received, then the bridge discards the packet. If no association is found in the station cache, the frame is forwarded to all ports except the inbound port.

Previous topic: Learning Bridge Source Address Processing on page 3
Next topic: Learning Bridge Station Cache Aging on page 4

5. Learning Bridge Station Cache Aging

Station cache aging for the Learning Bridge is controlled by an aging task, **tBridgeAger**.

This task wakes up periodically according to the value of AGING_CHECK_DELAY_SECONDS. The task checks to see if any entry in the table has reached AGING_TIMEOUT_SECONDS and removes any entries that have reached the timeout value from the station cache. When a packet is received from a media access control (MAC) address corresponding to a station cache entry, its age is reset.

> Note: The aging task does not remove static entries; they are permanent. A static entry represents a permanent mapping between the source MAC address and a bridge port.

The actual timeout value for any particular stale entry is somewhere between AGING_TIMEOUT_SECONDS and AGING_TIMEOUT_SECONDS + AGING_CHECK_DELAY_SECONDS. These constants are defined in **bridge.c**.

Previous topic: Learning Bridge Destination Address Processing on page 3
Next topic: Learning Bridge Packet Path Across the Bridge on page 4

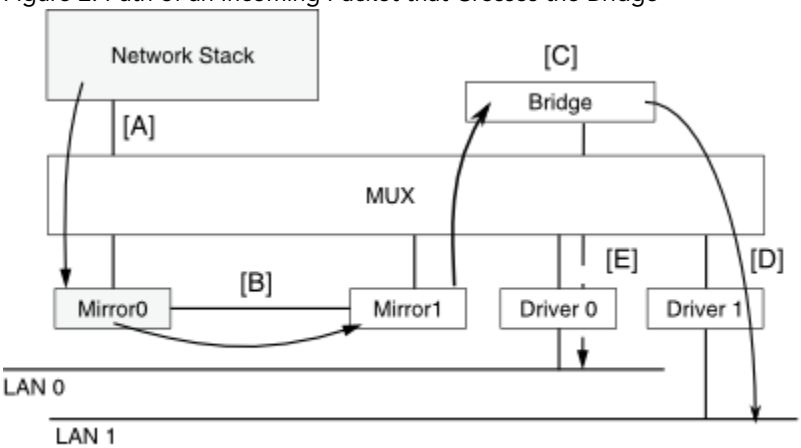
6. Learning Bridge Packet Path Across the Bridge

When developing an application using the Learning Bridge, you should understand the path of a packet that arrives at a driver on a LAN that is destined for a host on another LAN.

When Driver 1 accepts a packet from the network [A], it passes the packet to the multiplexer (MUX).

The bridge [B] is a service that is bound to Driver 1 as a SNARF protocol, so it sees the packet and determines it is destined for a host on LAN 0. It sends the packet out [C] through Driver 0, which is also bound to the bridge through the MUX.

Figure 2. Path of an Incoming Packet that Crosses the Bridge



If the bridge determines that the packet is not meant to cross the bridge (meaning it was destined for a host on the same LAN from which it originated), it drops the packet.

If the bridge does not know which LAN the packet is destined for (because it has not yet learned this), it sends the packet out to all the attached ports except for the port the packet arrived on (in this case, to both Driver 0 and **Mirror1**).

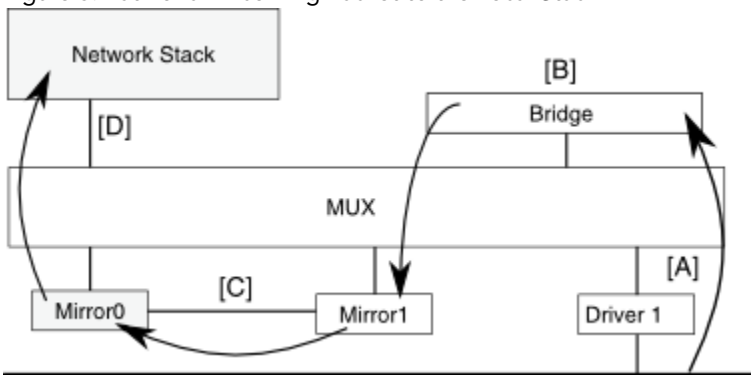
Previous topic: Learning Bridge Station Cache Aging on page 4
Next topic: Learning Bridge Packet Path Going To the Local Machine on page 5

7. Learning Bridge Packet Path Going To the Local Machine

Packets destined for the local machine are sent out on a pseudo-LAN.

Packets destined for the machine where the bridge is running are handled in an analogous manner, but the bridge sends the packet out on the pseudo-LAN by sending it to **Mirror1**.

Figure 3. Path of an Incoming Packet to the Local Stack



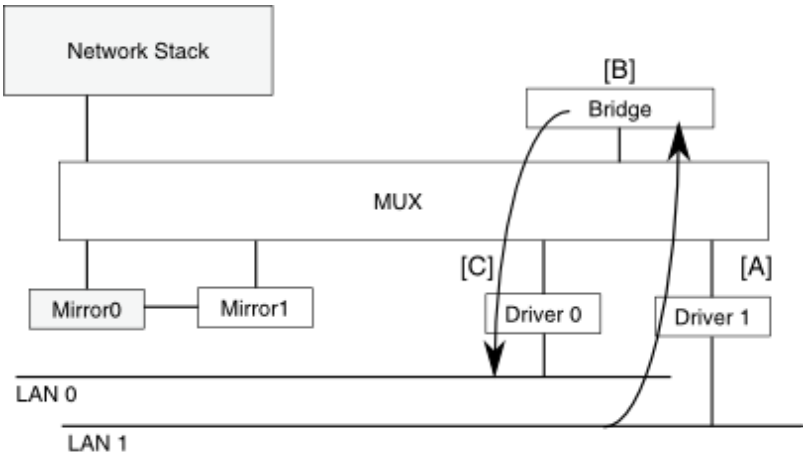
The figure above shows the path of a packet that arrives at a driver and is destined for the local stack. When Driver 1 accepts a packet from the network [A], it passes it to the multiplexer (MUX). The bridge [B] is a service that is bound to Driver 1 as a SNARF protocol, so it sees the packet and determines that the packet is destined for the local stack. It sends the packet out through **Mirror1**, which is also bound to the bridge through the MUX. All packets sent through **Mirror1** [C] are treated as received by **Mirror0**, although in fact the packet is neither sent nor received over the network. Upon receiving the packet, **Mirror0** passes it up to the stack [D] to which it is bound through the MUX.

Previous topic: Learning Bridge Packet Path Across the Bridge on page 4
Next topic: Learning Bridge Packet Path Leaving the Local Machine on page 5

8. Learning Bridge Packet Path Leaving the Local Machine

Packets sent by the stack that exist on the machine where the bridge is running are sent using the same pseudo-LAN that is used in the receive path.

Figure 4. The Path of Packets Leaving the Local Machine



The figure above shows the path of a packet that is being sent by the local stack to a host on LAN 1. The packet is first sent through the multiplexer (MUX) [A] to **Mirror0**, to which the local stack is bound. All packets sent though **Mirror0** [B] are treated as received by **Mirror1**, although in fact the packet is neither sent nor received over the network. Upon receiving the packet, **Mirror1** passes it up to the bridge, to which it is bound through the MUX. The bridge [C] determines that the packet is destined for a host on LAN 1, and sends the packet to Driver 1 [D], to which it is bound through the MUX.

If the bridge does not know which LAN the packet is destined for (because it has not yet learned this), it sends the packet out on all LANs (in this case, both LAN 0 through Driver 0 [E], and LAN 1 through Driver 1 [D]).

Previous topic: Learning Bridge Packet Path Going To the Local Machine on page 5
Next topic: Learning Bridge DHCP Relay Agent Information Option (Option 82) on page 6

9. Learning Bridge DHCP Relay Agent Information Option (Option 82)

The relay agent information option (option 82) feature enables dynamic host control protocol (DHCP) servers to implement security and IP address assignment policies based on information received by a bridge or relay agent. The DHCP server can also use this information to perform access control and set quality of service (QoS) for each client.

Security is enhanced with a shared key between the bridge and the server. IP address assignment can be based on the bridge input port identifier (circuit identifier) and/or a bridge identifier (remote identifier).

The DHCP server echoes the option 82 information back to the bridge in server-to-client replies. The bridge strips the option 82 information before forwarding the reply to the client.

The relay agent information option is organized as a single DHCP option that contains suboptions. The option and suboptions are as follows:

Relay agent information option (option 82) with the following suboptions:

A circuit identifier for the incoming port (suboption 1).

A remote identifier of a trusted remote device (suboption 2).

An authentication suboption containing a shared key used for data integrity and replay protection for relayed DHCP messages (suboption 8).

To configure the option 82 parameters with shell commands or from an application, see Learning Bridge Configuration with Shell Commands and API Functions Example on page 19.

Using the Circuit identifier Suboption Parameter

The circuit identifier is intended for use by bridges in relaying DHCP responses back to the proper port (client-side). The circuit identifier must be unique for a given bridge.

DHCP servers can use the circuit identifier for IP and other parameter assignment policies as they relate to a DHCP client.

Using the Remote identifier Suboption Parameter

You can add the remote identifier suboption to identify the remote host end of the circuit (server-side). Remote identifiers must be globally unique across a network.

Using the Authentication Suboption Parameter

You can add the authentication suboption to bridges that seek to ensure the integrity of the data they include in the relay agent option. A DHCP server configured to process the authentication suboption uses the information in the suboption to verify the checksum in the suboption and continues processing the relay agent information option only if the checksum is valid. If the DHCP server sends a response, it includes an authentication suboption in its response message. Bridges test the checksums in DHCP server responses to decide whether to forward the responses.

Option 82 Support for the Relay Agent and DHCP Server

For information on the relay agent information option for the Wind River relay agent and the Wind River DHCP server, see the VxWorks 7 DHCP Programmer's Guide.

Previous topic: Learning Bridge Packet Path Leaving the Local Machine on page 5
Next topic: Supported RFCs for Learning Bridge on page 7

10. Supported RFCs for Learning Bridge

Learning Bridge contains standards and design information defined by the Internet Engineering Task Force (IETF).

The Wind River Learning Bridge implements relay agent-specific features of RFC 3046: *DHCP Relay Agent Information Option*, and RFC 4030: *The Authentication Suboption for the Dynamic Host Configuration Protocol (DHCP) Relay Agent Option*.

Previous topic: Learning Bridge DHCP Relay Agent Information Option (Option 82) on page 6

2. CONFIGURATION AND BUILD

1. Learning Bridge Configuration and Build Workflow

Your Wind River VxWorks 7 installation includes source code but does not include pre-built kernel libraries.

Before beginning any VxWorks development, you must build the VxWorks kernel libraries (VxWorks Source Build (VSB) project). You must also build the VxWorks run-time kernel image based on the kernel libraries (VxWorks Image Project (VIP)).

For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the VxWorks 7 Configuration and Build Guide.

For a list of available source build configuration options for the learning bridge, see Learning Bridge Source Build Configuration Options (VSB) on page 8.

For a list of available kernel configuration components for the learning bridge, see Learning Bridge Kernel Configuration Components (VIP) on page 9.

For a list of available configuration parameters for the learning bridge, see Learning Bridge Configuration Parameters on page 9.

For most applications, default station cache settings are sufficient. For information on how to configure station chache parameters, see Learning Bridge Station Cache Parameters on page 10.

Next topic: Learning Bridge Source Build Configuration Options (VSB) on page 8

2. Learning Bridge Source Build Configuration Options (VSB)

Learning Bridge configuration includes VxWorks source build (VSB) options.

Name	Description
IPNET_BRIDGE	Enable learning bridge in the VSB.

Previous topic: Learning Bridge Configuration and Build Workflow on page 8
Next topic: Learning Bridge Optional Source Build Configuration Options on page 8

3. Learning Bridge Optional Source Build Configuration Options

Learning Bridge has some optional VxWorks source build (VSB) options.

Name	Description
IPNET_FIREWALL	Enable firewall support.
IPNET_DHCPR	Enable DHCP relay agent support.

Name	Description
COMPONENT_OPTION82_SUPPORT	Enable DHCP option 82 support for the bridge.

Previous topic: Learning Bridge Source Build Configuration Options (VSB) on page 8
Next topic: Learning Bridge Kernel Configuration Components (VIP) on page 9

4. Learning Bridge Kernel Configuration Components (VIP)

Learning Bridge configuration includes some VxWorks image project (VIP) kernel configuration components.

Name	Description
INCLUDE_BRIDGE	Include learning bridge. If you include and configure the learning bridge component into your project, the bridge startup function usrNetBridgeInit( ) is called automatically as part of VxWorks network initialization. The usrNetBridgeInit( ) function is defined in **usrNetBridgeCfg.c**.

Previous topic: Learning Bridge Optional Source Build Configuration Options on page 8
Next topic: Learning Bridge Optional Kernel Configuration Components on page 9

5. Learning Bridge Optional Kernel Configuration Components

You can modify the Learning Bridge behavior with a number of optional shell command components.

Name	Description
INCLUDE_IPCOM_SHELL_CMD	Include the IPCOM shell command interface to access the bridgeDHCPr shell command associated with the DHCP relay agent (INCLUDE_IPDHCPR) component.
INCLUDE_IPD_CMD	Include the daemon control shell command (ipd).
INCLUDE_IPCOM_SYSLOGD_CMD	Include the system logging shell command (syslog).

Previous topic: Learning Bridge Kernel Configuration Components (VIP) on page 9
Next topic: Learning Bridge Configuration Parameters on page 9

6. Learning Bridge Configuration Parameters

You can modify the Learning Bridge behavior with a number of configuration parameters and system variables.

Note: If you are not bridging IPv6 packets, set BRIDGE_IPV6_ADDR to NULL and BRIDGE_IPV6_PREFIXLEN to 0.

Parameter Name and Description	Default and Type
BRIDGE_IP_ADDR The IP address of the bridge. Specify a valid IP address enclosed within double-quotes; for example, "192.168.1.1".	**""** string
BRIDGE_IPV6_ADDR The IPv6 address of the bridge. Specify a valid IPv6 address enclosed within double-quotes; for example, "2001:DB8:1234:100::1".	**""** string
BRIDGE_IPV6_PREFIXLEN The IPv6 prefix length. Specify a valid IPv6 prefix length; for example, 64.	**0** int
BRIDGE_IP_MASK The IP mask of the bridge. Specify a valid IP mask; for example, 0xFFFFFF00.	**0** int
BRIDGE_PORTS A list of interfaces that are to be added as bridge ports. Each interface specified must include the device name and unit number. Specify a list of comma-separated interfaces enclosed within double-quotes; for example, "fei,1,fei,2"	**""** string
BRIDGE_AGING_TIMEOUT Bridge aging timeout time. Specify the bridge aging time, for example, 300.	**300** int

Previous topic: Learning Bridge Optional Kernel Configuration Components on page 9
Next topic: Learning Bridge Station Cache Parameters on page 10

7. Learning Bridge Station Cache Parameters

The **Learning Bridge** uses the station cache to keep track of associations between medi access control (MAC) addresses and bridge ports. There are three parameters that affect the behavior of the station cache. You can configure these parameters by editing **bridge.c**, changing these constants, and rebuilding your VxWorks source build (VSB) and VxWorks image project (VIP).

The three constants are:

STATION_CACHE_SIZE

This sets the maximum number of entries in the station cache. The station cache is configured to contain a maximum of STATION_CACHE_SIZE entries, which is 1024 by default. If this value is modified, the new station cache size must be a power of 2. This is because the station cache size is used to generate a bit mask in order to look up station cache entries.

AGING_CHECK_DELAY_SECONDS

This indicates how frequently the task that checks for stale cache entries wakes up. This value is expressed in seconds. The default is 15.

AGING_TIMEOUT_SECONDS

This indicates the amount of time after the last packet has been received from a particular MAC address that the entry for that MAC address will be removed from the station cache. Note that static entries are not removed; they are permanent. This value is expressed in seconds. The default is 300 (five minutes). The actual timeout value for any particular stale entry will be somewhere between AGING_TIMEOUT_SECONDS and (AGING_TIMEOUT_SECONDS + AGING_CHECK_DELAY_SECONDS), therefore between 300 and 315 seconds by default.

Previous topic: Learning Bridge Configuration Parameters on page 9

3. SHELL COMMANDS

1. Obtaining the Version of the Learning Bridge

After you have booted your target, you can verify the version of Learning Bridge by examining the value of the *bridgeVersion* variable.

About This Task

For example, to print this value directly to a running VxWorks target shell, enter the following command:

```
-> printf("Learning Bridge version: %s\n", bridgeVersion);
Learning Bridge version: 1.3.3
value = 31 = 0x1f
```

Next topic: Learning Bridge bridgeDHCPr Shell Command on page 12

2. Learning Bridge bridgeDHCPr Shell Command

The bridgeDHCPr C shell command allows you to set and list the relay agent parameters for information option 82.

To get help for the bridgeDHCPr C shell command, use the following command:

```
-> bridgeDHCPr
```

The command syntax is as follows:

```
bridgeDHCPr "option82 {enable | disable} [auth | drop]"

bridgeDHCPr "port { devname} {circuit-id id-string | remote-id id-string}"
bridgeDHCPr "port {devname} {{enable auth auth-string} | {disable auth}}"
bridgeDHCPr "port {devname} trust {enable | disable}"

bridgeDHCPr "list [devname | option82]"
```

option82

Controls the state of the option 82 feature (enabled or disabled), and specifies the authentication and the drop policy for the learning bridge. The available command settings are as follows:

- enable enables the relay agent information option (option 82) for the learning bridge. The relay agent information option functionality is disabled by default.
- disable disables the relay agent information option (option 82) for the learning bridge.
- enable auth enables option 82 authentication with the DHCP server using a shared key. Authentication is disabled by default.
- disable auth disables option 82 authentication with the DHCP server.

- enable drop enables the option 82 drop policy. When the drop policy is enabled and a packet is received with the proper authentication (shared-key), the packet is processed; otherwise the packet is dropped. The drop policy is disabled by default.
- disable drop disables the option82 drop policy. When the drop policy is disabled, packets are processed by the relay agent without validating the option 82 authentication (shared key).

port

Configures the port parameters for the relay agent information option. The *auth-string* is a shared key for the port.

The **circuit-id** is an identifier for the incoming port of the bridge. The default value is **NULL**.

The **remote-id** is a remote identifier of a trusted remote device. The default value is **NULL**.

When **enable auth** is set for a port, the shared key must be included. Authentication is disabled by default. The default shared key is **NULL**.

When **disable auth** is set for a port, option 82 authentication between the bridge and the DHCP server is not enforced.

When **trust enable** is set for a port, all packets received at the port are assumed to be from trusted sources and are forwarded. Trust is disabled by default.

When **trust disable** is set for the port, all packets received at the port are assumed to be from untrusted sources. The received packets at the port are verified against the option 82 configuration parameters. If the packet contains the necessary option 82 information, the packets are forwarded, otherwise the packet is dropped.

list

Lists the parameter values of a device or of the relay agent information option.

Previous topic: Obtaining the Version of the Learning Bridge on page 12

4. API FUNCTIONS

1. About the Learning Bridge API

The Learning Bridge API is composed of several libraries.

For full details on the API functions, see the bridge, mirrorEnd, and mirrorUtils API reference entries. The learning bridge API functions are defined in bridge.c, mirrorEnd.c, and mirrorUtils.c.

- **bridge** - simple learning bridge without STP
- **mirrorEnd** - mirroring END device driver
- **mirrorUtils** - utility functions for the mirror END driver

Bridge API Library on page 14 The Bridge library implements a simple learning bridge without STP (Spanning Tree Protocol). The Learning Bridge API is composed of the **bridge**,**mirrorEnd** and **mirrorUtils** libraries.
MirrorEnd API Library on page 16 The Mirror END library implements a mirroring END device driver. It consists of a single function, mirrorEndLoad( ).
MirrorUtils API Library on page 16 The Mirror END driver utility library contains the utility functions for implementing the mirror END driver.

1.1. Bridge API Library

The Bridge library implements a simple learning bridge without STP (Spanning Tree Protocol). The Learning Bridge API is composed of the **bridge**,**mirrorEnd** and **mirrorUtils** libraries.

Learning Bridge API Functions

Function	Description
bridgeDHCPr( )	Set and list the relay agent information option parameters. For information on how to configure the option parameters from the command shell, see 3.2 bridgeDHCPr Shell Command, p. 15.
bridgeInit( )	Initialize the bridge. This function must be called before you use any of the other bridge functionality.
bridgeMacAdd( )	Add a MAC address to the bridge station cache.
bridgeMacRemove( )	Remove a MAC address from the bridge station cache.
bridgePortAdd( )	Add a device as a bridge port.
bridgePortRemove( )	Remove a bridge port.
bridgePortListShow( )	Show a list of all bridge ports (diagnostic function).
bridgeShutdown( )	Shut down the bridge.

Function	Description
bridgeStationCacheShow( )	Display all entries in bridge station cache (diagnostic function).

Note: The bridgeInit( )and bridgePortAdd( ) functions are provided for reference only. Wind River does not recommend calling them directly for initializing the bridge. Initialization should be done using usrNetBridgeInit( ). This function is called automatically if you use Workbench or wrtool to configure the Wind River Learning Bridge. The usrNetBridgeInit( ) function is in the **usrNetBridgeCfg.c** file.

bridgeDHCPr( )

This function allows you to configure and display the relay agent information option 82 parameters. The default configuration is as follows:

- the relay agent information option functionality is disabled
- authentication is disabled
- the drop policy is disabled
- the remote identifier is not defined
- the circuit identifier is not defined
- the port trust state is disabled
- the shared key is not defined

For examples on how to enable the relay agent information option and configure the option 82 parameters using the bridgeDHCPr( ) ( )API function and the bridgeDHCPr shell command, see Figure 5-4Example Configuration with Shell Commands and API Functions, p. 24.

bridgeInit( )

This function initializes the bridge and station cache, and creates a bridge port list and station cache access mutexes.

If DHCP relay agent support is included, this function initializes the bridge port DHCP relay agent information list.

If firewall support is included, this function allocates resources for a firewall packet header.

This function must be called before you use any of the other bridge functionality.

bridgeMacAdd( )

This function provides the ability to add a MAC address to the bridge station cache directly instead of waiting for the bridge to automatically learn it. This function is typically used to add a static entry to the station cache.

A static entry in the station cache represents a permanent mapping between a source MAC address and a bridge port. Static entries are useful if you already know the MAC address to bridge port mapping for your interfaces and want to set these at initialization time instead of waiting for the bridge to learn them. Static entries also prevent the MAC address of an interface from migrating from one port to another, as can happen if you have a network loop. If the migration happens, the bridge may stop receiving packets.

bridgeMacRemove( )

This function provides the ability to remove a MAC address from the bridge station cache directly. This function is typically used to remove a static entry from the station cache.

bridgePortAdd( )

This function adds a device as a bridge port to the bridge port list. Using the example shown in Figure 1-1, bridgePortAdd( ) is called for the interfaces corresponding to **Mirror1** and **Driver 0** through **Driver N**. This function is called for the network interfaces that are attached to the bridge as ports, including **Mirror1**. This function binds the port's device driver through the MUX as a network service with type **MUX_PROTO_SNARF**.

bridgePortRemove( )

This function removes an existing port from the bridge port list and unbinds the bridge service from the MUX layer for that port.

bridgePortListShow( )

This function displays the contents of the bridge port information list.

bridgeShutdown( )

This function performs the shutdown procedure of the bridge. It deletes the bridge aging task, removes the bridge protocol interface from MUX layer, reverts the devices to non-promiscuous mode, unloads the mirror devices, and frees all allocated resources. Once this function is called, the bridge can be restarted using its standard initialization function.

Do not call this function concurrently with the bridgePortRemove( ) and bridgePortAdd( ) functions. No data packet should be sent to the bridge ports by the local IP stack during the operation.

bridgeStationCacheShow( )

This function displays the contents of the bridge station cache. The cache contains what the bridge has learned and remembered about the mapping of MAC addresses and bridge ports.

Parent topic: About the Learning Bridge API on page 14

1.2. MirrorEnd API Library

The Mirror END library implements a mirroring END device driver. It consists of a single function, mirrorEndLoad( ).

mirrorEndLoad( )

This function initializes the driver and the device. All of the device-specific parameters are passed in the **initString** parameter. **initString** is of the following format, where *unit* is the device driver unit number:

unit:

Mirror driver unit number can be MIRROR_STACK_UNIT_NUM (0) or MIRROR_BRIDGE_UNIT_NUM (1). These are defined in **mirrorEnd.h**. Typically, unit 0 is attached to the network stack and unit 1 is attached to the bridge.

This function returns an END object pointer on success, or NULL on error.

Parent topic: About the Learning Bridge API on page 14

1.3. MirrorUtils API Library

The Mirror END driver utility library contains the utility functions for implementing the mirror END driver.

mirrorEnetAddrGet( )

This function copies the MAC address from the END object of the interface specified. The data is copied into the buffer provided to this function.

mirrorBaseParamsSet( )

This function stores the base device name and unit number. It is needed for mirrorEnetAddrGet( ) to work.

Parent topic: About the Learning Bridge API on page 14

5. APPLICATIONS

1. Learning Bridge Configuration and Test Example

An example learning bridge configuration with three ports:

About This Task

- The gateway has three interfaces: **fei0**, **fei1**, and **fei2**.
- All three interfaces are bridged.
- The IP address of the bridge is 192.168.1.1.
- The IP mask of the bridge is 0xFFFFFF00.

For this example, complete the following:

Procedure

1. Include the INCLUDE_BRIDGE component.
2. Set the BRIDGE_IP_ADDR parameter to "*192.168.1.1* " (include the double-quotes).
3. Set the BRIDGE_IP_MASK parameter to *0xFFFFFF00* (no quotes).
4. Set the BRIDGE_PORTS parameter to "*fei,0,fei,1,fei,2* " (include the double-quotes).

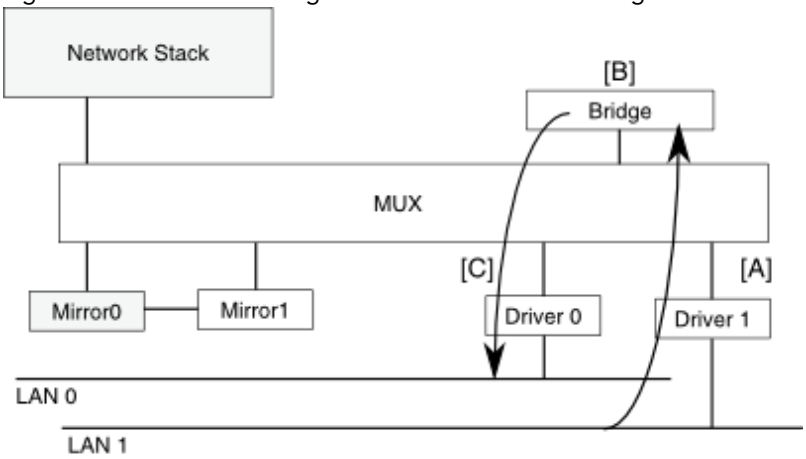
Confirm the Bridge is Working on page 18

Next topic: Learning Bridge Configuration with Shell Commands and API Functions Example on page 19

1.1. Confirm the Bridge is Working

- Send traffic across the bridge. For example, in the figure below successfully ping host on Network [A] from host on Network [C] and vice-versa.
- Send traffic to the bridge. For example, in the figure below successfully ping Bridge [B] from Network [A] and [C].
- Send traffic from the bridge. For example, in the figure below successfully ping host on Network [A] and [C] from Bridge [B].

Figure 5. Path of an Incoming Packet that Crosses the Bridge



Parent topic: Learning Bridge Configuration and Test Example on page 18

2. Learning Bridge Configuration with Shell Commands and API Functions Example

Examples showing how to configure the relay agent information option parameters using the bridgeDHCPr( )API function and the bridgeDHCPr C shell command.

The default configuration is as follows:

- the relay agent information option functionality is disabled
- authentication is disabled
- the drop policy is disabled
- the remote identifier is not defined
- the circuit identifier is not defined
- the port trust state is disabled
- the shared key is not defined

Enable the Relay Agent Information Option Functionality for the Bridge.

```
-> bridgeDHCPr "option82 enable"
```

The API method is as follows:

```
status = bridgeDHCPr ("option82 enable");
```

Enable Relay Agent Information Option Authentication.

```
-> bridgeDHCPr "option82 enable auth"
```

The API method is as follows:

```
status = bridgeDHCPr ("option82 enable auth");
```

Enable the Relay Agent Information Option Drop Policy.

```
-> bridgeDHCPr "option82 enable drop"
```

The API method is as follows:

```
status = bridgeDHCPr ("option82 enable drop");
```

Configure a Remote Identifier for the fei1 Port.

```
-> bridgeDHCPr "port fei1 remote-id bridge_id_1"
```

The API method is as follows:

```
status = bridgeDHCPr ("port fei1 remote-id bridge_id_1");
```

Configure a Circuit Identifier for the fei1 Port.

```
-> bridgeDHCPr "port fei1 circuit-id test_users"
```

The API method is as follows:

```
status = bridgeDHCPr ("port fei1 circuit-id test_users");
```

Configure the Trust State for Port fei1.

```
-> bridgeDHCPr "port fei1 trust enable"
```

The API method is as follows:

```
status = bridgeDHCPr ("port fei1 trust enable");
```

Configure the Shared Key Shared with the DHCP Server for Bridge Port fei1.

```
-> bridgeDHCPr "port fei1 enable auth 5har3d-k3y"
```

The API method is as follows:

```
status = bridgeDHCPr ("port fei1 enable auth 5har3d-k3y");
```

Obtain the Configuration for Port fei1.

```
-> bridgeDHCPr "list fei1"
----------------------------------------------
devname    : fei1 trust : disabled
sndRDcount : 0 rcvRDcount : 0
circuitID  : NULL
remoteID   : NULL
shareKey   : NULL
value = 0 = 0x0

-> bridgeDHCPr "list option82"
option82 is disabled.
option82 authentication is disabled.
option82 drop policy is disabled.
```

The API method is as follows:

```
status = bridgeDHCPr ("list fei1");
```

Obtain the Relay Agent Information Option Configuration for Port fei1.

```
-> bridgeDHCPr "list option82"
option82 authentication is enabled.
option82 drop policy is enabled.
```

The API method is as follows:

```
status = bridgeDHCPr ("list option82");
```

Previous topic: Learning Bridge Configuration and Test Example on page 18